

INTRODUCTION TO

MULTIPLE

CLOUD COMPUTING

DEV – DEVELOPMENT

SEC – SECURITY

OPS – OPERATION

WITH

ARTIFICIAL INTELLIGENCE

INTEGRATION

Development Security Operations

Single Cloud Engineering => Multi Cloud Engineering + Dev Sec Ops with AI

To avoid Vendor Lock - in
Best services from each cloud
Disaster Recovery

Two Clouds and Three CI CD Pipelines [Git Hub Actions]

Cloud + Scripting + CICD	Cloud + Scripting + CICD
Azure + Terra Form (IAC) + Azure Dev Ops	AWS + Terra form + Jenkins

Dev Ops : Git, Github, Docker, Kubernetes, Ansible, Prometheus, Grafana, ELK, Maven

Security : Sonar, Trivy, Gitleaks, Checkov

Scripting Languages : Linux basics, PowerShell, Bash, Python for DevOps

Drive Link :

Vcube_B28-Offline_MultiCloudDevSecOps_9:30AM

https://drive.google.com/drive/folders/1ncwMUzjQ2R_yEIF4a2a7H8UScoTLbNc

MULTICLOUD-DEVSECOPS-DEMO-OCTOBER-25

<https://drive.google.com/drive/folders/1iILfSAMdXpPQIhNab5rFsVK5QVPMnwyj>

1. CLOUD COMPUTING

1.1 What is Data Center

- A data center is a physical facility that houses an organization's critical IT infrastructure, such as servers, storage systems, and networking equipment, to store, process, and manage data.
- These facilities are designed to ensure the security, reliability, and availability of applications and data, often with redundant components for power and cooling

Or



- A data center is a secure, climate-controlled building (or part of a building) designed to:
 - Store digital information (data)
 - Run applications
 - Support online services (like Google, Netflix, banking apps, etc.)
 - Provide redundancy (backup power, cooling, and connectivity to ensure uptime)

➔ Parts of a Data Center

I. IT Equipment

Servers – Powerful computers that store and process data and applications.

Storage Systems – Devices like HDDs, SSDs, or SAN (Storage Area Network) used to store data.

Networking Equipment – Routers, switches, firewalls, and cables that connect servers and users to the internet.

II. Power Systems

Power Supply Units (PSUs) – Provide power to servers and other devices.

Uninterruptible Power Supplies (UPS) – Backup batteries that keep systems running during short power outages.

Generators – Provide long-term backup power during extended outages.
Power Distribution Units (PDUs) – Distribute power efficiently to racks and servers.

III. Cooling Systems

Computer Room Air Conditioning (CRAC) or Air Handling Units (AHUs) – Maintain optimal temperature and humidity.

Chillers and Cooling Towers – Help remove excess heat from the building.

Raised Floors or Hot/Cold Aisle Design – Manage airflow and heat dissipation.

IV. Network Infrastructure

Fiber and Ethernet Cabling – Connect internal and external networks.

Internet Connectivity – High-speed connections to the global internet backbone.

Load Balancers – Distribute network traffic evenly across multiple servers.

V. Security Systems

Physical Security – Cameras, biometric access controls, and guards.

Cybersecurity – Firewalls, intrusion detection systems, and encryption.

VI. Monitoring and Management

DCIM (Data Center Infrastructure Management) tools to monitor power, temperature, and performance.

Network Monitoring Software to detect failures or bottlenecks.

VII. Building and Infrastructure

Racks and Cabinets – Hold and organize servers and network devices.

Fire Suppression Systems – Protect equipment from fire damage.

Cabling Systems – Structured cable layouts for organization and performance.

➔ There are 2 types of Data Center

1.1.1 On Premises Data Center

➔ On-premise data center is a private infrastructure that a company owns and operates within its own facilities, giving it full control over all hardware, software, and data.

➔ Companies choose this model for direct oversight, customization, and compliance with regulations, but the organization is responsible for all maintenance, power, and cooling

Pros (Advantages)	Cons (Disadvantages)
Full control over hardware, data, and security.	Very expensive to build and maintain.
Customizable to the company's needs.	Needs skilled IT staff to manage it.
Data stays on-site — better privacy.	Limited scalability (hard to expand quickly).
No dependency on internet for internal access.	Power or hardware failures can cause downtime.
Easier to follow strict security or legal rules.	High energy use (power and cooling costs).

1.1.2 Cloud Data Center

➔ A cloud data center is a remote, outsourced facility run by a cloud service provider that hosts and manages IT infrastructure and resources like servers, storage, and networking.

- ➔ Instead of an organization owning and maintaining its own on-premise data center, it leases these resources over the internet, allowing for on-demand scalability, flexibility, and a pay-per-use model for services such as computing power, storage, and applications.

Pros (Advantages)	Cons (Disadvantages)
Easy to scale up or down as needed.	Depends on a stable internet connection.
Lower upfront cost — pay only for what you use.	Less control over hardware and data location.
No need to maintain physical servers.	Ongoing monthly or usage costs can add up.
Accessible from anywhere.	Possible security and privacy concerns.
Fast setup and easy to manage.	Data transfer between clouds can be slow or costly.

- ➔ Difference between On – Premises Data Center and Cloud Data Center

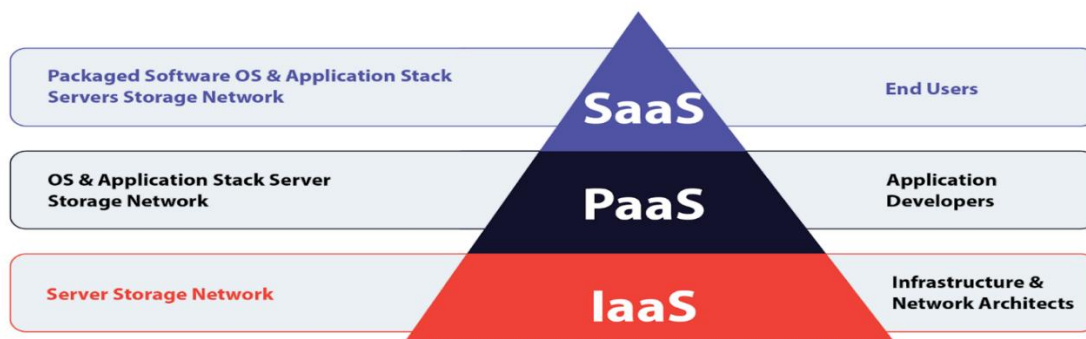
Feature	On-Prem Data Center	Cloud Data Center
Ownership	Owned and managed by the organization	Owned and managed by cloud provider
Infrastructure Cost	High upfront (CapEx)	Pay-as-you-go (OpEx)
Scalability	Limited, manual hardware upgrades	Highly scalable, on-demand
Maintenance	Handled by in-house IT team	Handled by cloud provider
Deployment Time or Setup Time	Slow (weeks to months)	Fast (minutes to hours)
Accessibility	Local network access	Accessible via internet globally
Security Control	Full control by organization	Shared responsibility with provider
Flexibility	Less flexible, hardware-bound	Highly flexible, supports rapid changes
Disaster Recovery	Complex and costly	Built-in and easily configurable
Updates/Upgrades	Manual and periodic	Automatic and continuous

DAY 2

1.2 What is Cloud Service Model

- A cloud service model is a category of cloud computing that specifies the level of control, management, and responsibility between the cloud service provider and the user.

Cloud Service Models



Cloud service is offering 3 Models

- I. IaaS - Infrastructure as a Service
- II. PaaS - Platform as a Service
- III. SaaS - Software as a Service

I. IaaS - Infrastructure as a Service :

- A cloud computing model that provides on-demand access to virtualized computing resources like servers, storage, and networking over the internet on a pay-as-you-go basis.
- It allows businesses to rent IT infrastructure instead of buying and managing their own hardware, offering flexibility, scalability, and cost savings.
- The cloud provider handles the underlying physical infrastructure, while the customer is responsible for the operating system, applications, and data.

In Simple:

- The provider manages: servers, networking, and storage.
- You manage: OS, runtime, data, and applications.

Advantages and Dis advantages

Pros (Advantages)	Cons (Disadvantages)
Cost-effective – no need to buy hardware	Security depends partly on the provider
Easy to scale up or down	Needs skilled IT staff to manage
Quick to set up and deploy	Possible downtime if provider fails
Pay only for what you use	Costs can rise if not monitored
Access from anywhere via internet	Less control over physical servers
Reliable with backup and recovery options	Can lead to vendor lock-in (hard to switch providers)

Ex : of IaaS Providers: all provide virtual servers, storage, and networking on demand.

Amazon Web Services (AWS) – EC2, S3 storage, VPC networking

Microsoft Azure – Virtual Machines, Azure Storage Blob and Virtual network

Google Cloud Platform (GCP) – Compute Engine VMs, Cloud Storage and VPC

IBM Cloud, Oracle Cloud Infrastructure (OCI)

IaaS : Infrastructure as a service : Customization is available

Ex : RAW for a rent

Linux / Windows => download java, install, configure, depend, app deploy => our team

II. PaaS - Platform as a Service

- A is a cloud computing model that provides a complete platform for developing, running, and managing applications
- Without needing to manage the underlying infrastructure, like servers, operating systems, or hardware.
- It offers a ready-to-use environment with development tools, databases, and other services, allowing development teams to focus on coding and deploying applications more quickly and cost-effectively.

In Simple

- The provider manages: infrastructure, OS, and runtime environment.
- You manage: applications and data.

➔ Advantages and Disadvantages

Pros (Advantages)	Cons (Disadvantages)
Easy and fast app development	Less control over the underlying infrastructure
No need to manage servers or hardware	May cause vendor lock-in (hard to switch providers)
Cost-effective — pay only for what you use	Limited customization options
Scalable — can grow with your app	Security depends partly on the provider
Includes built-in tools and frameworks	Not all platforms support every programming language
Focus on coding, not maintenance	Downtime or outages affect your app availability

PaaS Ex :

Google App Engine – for building and running web and mobile apps

Microsoft Azure App Service – for web apps, APIs, and mobile backends

AWS Elastic Beanstalk – deploy applications with automatic scaling

IBM Cloud Foundry – platform for cloud-native app development

Red Hat OpenShift – container-based app development and deployment

Paas : Platform as a service : Customization is not available

Ex : Studio room in a hotel

Pre installed and pre configured machine

Coding lang with all versions

III. SaaS – Software as a Service

- A software delivery model where applications are hosted by a provider and made available to users over the internet, typically through a web browser.
- This allows users to access software on a subscription basis
- Without needing to install or manage it on their own devices,
- As the provider handles infrastructure, maintenance, and updates.

In Simple :

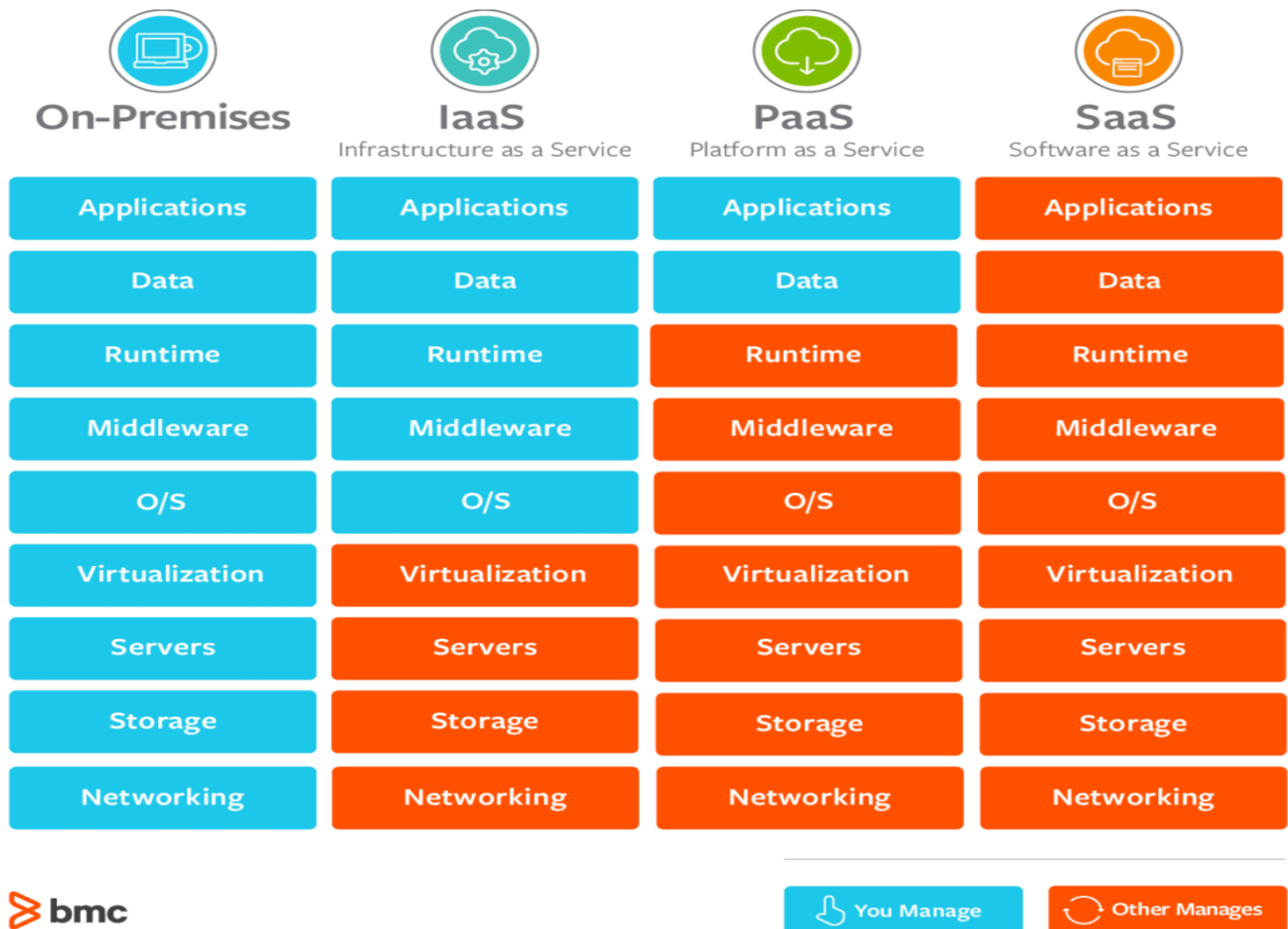
- You just use the software.
- The provider manages: everything (infrastructure, app, and data storage).

Examples for SaaS Model

- Microsoft Azure (Microsoft SaaS) : Microsoft 365, Dynamics 365, Power BI.
- Amazon Web Services (AWS SaaS) : Amazon Chime, Amazon Work Mail, Amazon Work Docs.
- Google Cloud Platform (GCP SaaS) : Google Workspace, Google Maps Platform, Google Analytics

There are more Extended or Specialized Cloud Service models but rare to use and study

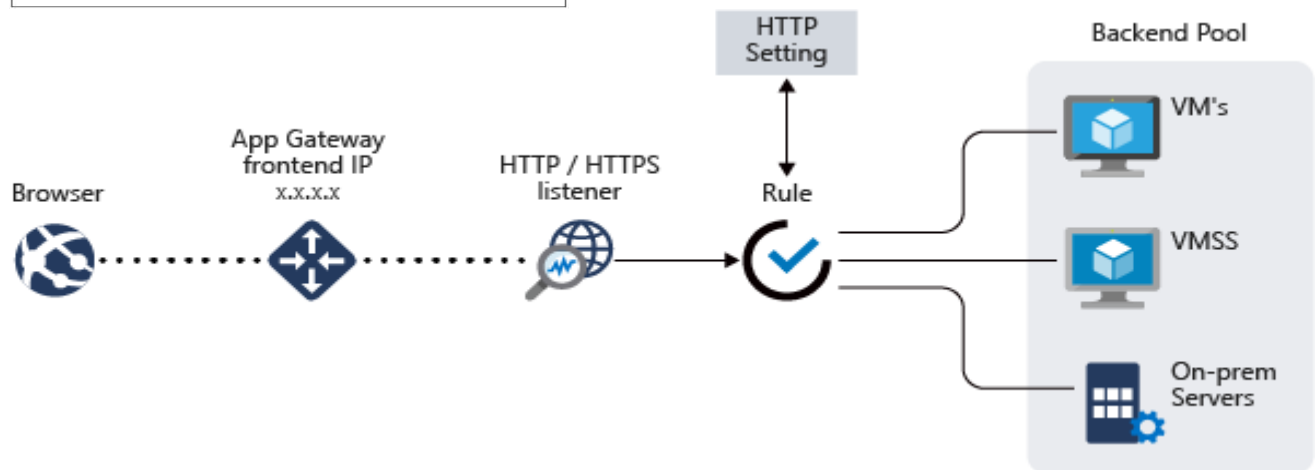
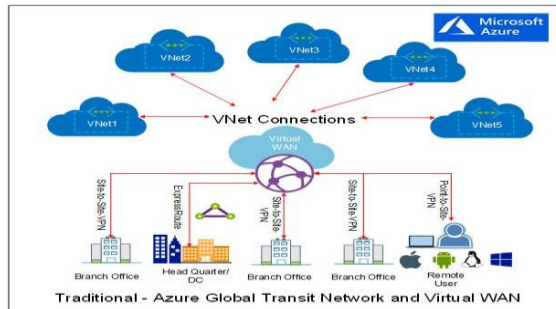
- IV. FaaS (Function as a Service) – Run code on demand without managing servers (serverless).
- V. DaaS (Desktop as a Service) – Access virtual desktops via the cloud.
- VI. BaaS (Backend as a Service) – Cloud backend services for mobile/web apps.
- VII. CaaS (Containers as a Service) – Manage and run containerized applications.
- VIII. DBaaS (Database as a Service) – Cloud-managed databases.



1.3 BASIC NETWORKING

- ➔ **Basic networking** refers to the foundational concepts, devices, and rules that allow computers and other devices to communicate with each other.
- ➔ It is the “language” and “infrastructure” that enables data to travel from one point to another—whether across a room or across the world.
- ➔ A **network** is a group of devices (computers, phones, servers, routers, etc.) connected to share information and resources.
- ➔ Networking follows rules called protocols.
The most important is TCP/IP (Transmission Control Protocol / Internet Protocol).
 - IP: Identifies a device (like a home address).
 - TCP: Ensures reliable delivery of data.
- ➔ **Common Networking Terms**
 - **DNS:** Converts website names into IP addresses.
 - **MAC Address:** Unique hardware address of a network device.

- **Bandwidth:** Maximum data rate.
- **Latency:** Time it takes for data to travel.
- **Subnetting:** Dividing a network into smaller parts.



❖ Basic networking = Devices + Connections + Rules for communication.

1.3.1. IP ADDRESS

- ➔ An **IP address** (short for **Internet Protocol address**) is a unique numerical identifier assigned to each device that connects to a network (like the internet or a private LAN).
- An **IP address** identifies a device (such as a computer, phone, or server) on a network.
 - It ensures that information sent over the internet (like emails, web pages, or files) reaches the **correct destination**.
 - IP addresses are defined and managed by the **Internet Protocol (IP)** — a core communication protocol of the internet.
 - It functions like a **“home address”** for your device allowing it to **send and receive data** across a network.

IP Address

[eye-pee uh-dres]

An IP address is a unique identifier for devices on a network, enabling communication and data transfer between them.



Ex Examples.com

Types

- Public IP Addresses
- Private IP Addresses
- Static IP Addresses
- Dynamic IP Addresses
- IPv4 Addresses
- IPv6 Addresses
- Loopback IP Addresses
- Link-local Addresses
- Reserved IP Addresses

2.1.1 Structure of an IP Address

An IP address is made up of a **series of numbers**, separated by dots or colons, depending on the version. There are **two versions** in use:

1. IPv4 (Internet Protocol version 4)	2. IPv6 (Internet Protocol version 6)
Most common version (though slowly being replaced by IPv6).	Designed to replace IPv4 and provide many more addresses .
Uses 32 bits, written as four decimal numbers separated by dots.	Uses 128 bits , written as eight groups of hexadecimal numbers separated by colons.
Example: 192.168.1.10	Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334
Each number can range from 0 to 255.	Abbreviations allowed (to simplify notation): 2001:db8:85a3::8a2e:370:7334
Total possible unique addresses: $2^{32} = \sim 4.3$ billion addresses.	Total possible unique addresses: $2^{128} \approx 3.4 \times 10^{38}$ addresses — virtually unlimited.

Problem: The world ran out of IPv4 addresses due to the explosion of internet-connected devices. IP address can not be defined ranges without CIDR as it divides it

➔ Types of IP Addresses

IP addresses can be categorized in multiple ways:

I. Based on Scope

Type	Description	Example
Public IP Address	Used on the internet ; globally unique; assigned by ISPs (Internet Service Providers) .	8.8.8.8 (Google DNS)
Private IP Address	Used within private networks (like home or office LANs); not routable on the internet.	192.168.x.x, 10.x.x.x, 172.16.x.x – 172.31.x.x

Private IPs communicate inside the network;
A router translates them to a public IP using **NAT (Network Address Translation)** to access the internet.

II. Based on Stability

Type	Description	Example Use
Static IP Address	Manually assigned, doesn't change; ideal for servers, websites, or devices needing a consistent address.	Web servers, DNS servers
Dynamic IP Address	Automatically assigned by DHCP (Dynamic Host Configuration Protocol) and may change over time.	Home computers, mobile devices

2.1.2 What Is an IP Address Class?

➔ An **IP address class** is a way of categorizing IPv4 addresses based on how many **network bits** and **host bits** they contain.

➔ Each class defines:

- How many **networks** can exist.
- How many **hosts (devices)** each network can support.

➔ IP address classes are also called or known as **Class full addressing**

➔ IPv4 Address Basics

- IPv4 address = **32 bits** (divided into 4 octets)
- Written as: **x.x.x.x** (e.g., 192.168.1.1)
- Each octet = 8 bits → value range 0–255
- The **first few bits** of the address determine the class.

➔ IP Address Classes and Their Ranges

Class	First Octet Range	Default Subnet Mask	Network / Host Distribution	Example Address	Usage
A	1 – 126	255.0.0.0 (/8)	8 bits for network, 24 for host	10.0.0.1	Very large networks (ISPs, big corps)
B	128 – 191	255.255.0.0 (/16)	16 bits for network, 16 for host	172.16.5.4	Medium networks (universities, enterprises)
C	192 – 223	255.255.255.0 (/24)	24 bits for network, 8 for host	192.168.1.10	Small networks (home, offices)
D	224 – 239	N/A	Used for multicasting	224.0.0.1	Group communication
E	240 – 255	N/A	Reserved for research and experimental use	245.1.2.3	Research only

Special Notes

- **127.x.x.x** → Reserved for **loopback** (local testing).
Example: 127.0.0.1 = "localhost"
- **0.x.x.x** → Represents "this network"
- **255.255.255.255** → **Broadcast address**

➔ Private IP Ranges by Class (RFC 1918)

Class	Private Range	CIDR Notation	# of IPs
A	10.0.0.0 – 10.255.255.255	10.0.0.0/8	16,777,216
B	172.16.0.0 – 172.31.255.255	172.16.0.0/12	1,048,576
C	192.168.0.0 – 192.168.255.255	192.168.0.0/16	65,536

- Rest IP address not in of private are all public IP address

Quick Reference Summary

Class	Range (1st Octet)	Subnet Mask	Networks	Hosts per Network	Use Case
A	1–126	255.0.0.0	128	16 million	Very large networks
B	128–191	255.255.0.0	16,384	65,000	Medium networks
C	192–223	255.255.255.0	2 million	254	Small networks
D	224–239	N/A	N/A	N/A	Multicasting
E	240–255	N/A	N/A	N/A	Experimental

Some imp Points

- For changing Private to public or converting NAT is responsible
- Internet Service Provider is responsible public IP provider
- Private is used locally or internal hence it is used same IP at different places
- Because not exposed for public
- But public is different and unique

1.3.2. CIDR - CLASSLESS INTER-DOMAIN ROUTING

➔ CIDR stands for Classless Inter-Domain Routing.

- It's a method used to allocate IP addresses and route IP packets more efficiently.
- CIDR replaced that with variable-length subnet masks, allowing networks to use exactly as many IP addresses as they need.
- CIDR is a way to allocate and represent IP addresses efficiently using a slash notation instead of old fixed IP classes.
- CIDR = flexible IP addressing using slash notation, used for subnetting, routing, and cloud networking

➔ IP address consists of 4 octets = 32 bits

1 octet = 8 bits

- How CIDR looks :-

Example:

192.168.1.0/24

192.168.1.0 → Network address

/24 → Number of network bits (subnet mask)

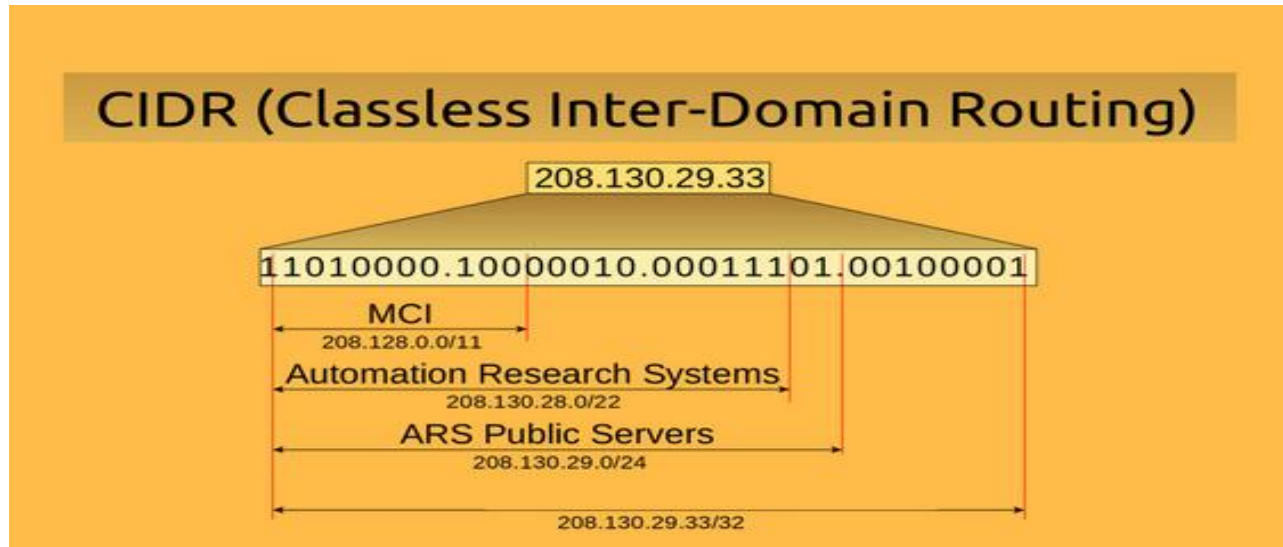
/24 means:

Subnet mask = 255.255.255.0

Usable IPs = 254

➔ Why CIDR is used :

- Before CIDR, IPs were divided into rigid Class A, B, C → lots of IPs got wasted 😬
- CIDR fixes that by allowing flexible subnet sizes.



➔ CIDR Notation

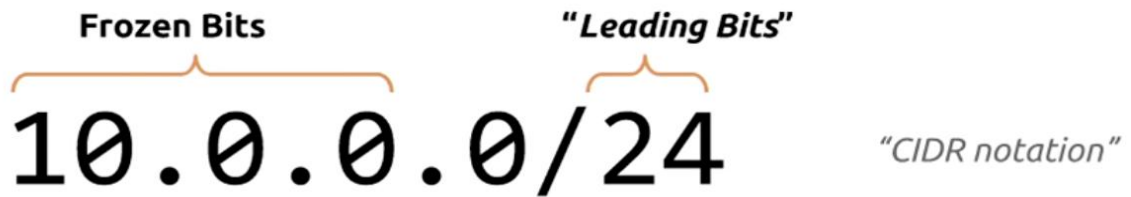
CIDR notation looks like this:

192.168.1.0/24

- 192.168.1.0 → the network address
- /24 → the prefix length, meaning the first 24 bits are the network part
- the remaining (32 – 24 = 8) bits are for hosts.

So /24 corresponds to the subnet mask 255.255.255.0.

CIDR Notation	Subnet Mask	# of IP Addresses	Usable Hosts
10.0.0.0/8	255.0.0.0	16,777,216	16,777,214
172.16.0.0/16	255.255.0.0	65,536	65,534
192.168.1.0/24	255.255.255.0	256	254

Frozen Bits
"Leading Bits"

 10.0.0.0/24


 10.0.0.0 – 10.0.0.255

256 addresses

➔ Why CIDR Matters

- Makes IP address allocation more flexible
- Reduces waste of IPv4 addresses
- Improves routing efficiency by aggregating multiple routes (called route summarization or super netting)

CIDR range decrease , IP address increases, Hosts decreases

CIDR range increases, IP address decreases, Hosts increases]]]

➔ How CIDR is used (real life) :

- Subnetting : Divide a network into smaller networks
Example : /16 → /24 → /28
- Efficient IP usage : Allocate only the number of IPs you actually need
- Routing (Internet & Cloud) : Routers use CIDR to summarize routes
Example : 10.0.0.0/8 represents millions of IPs in one route
- Cloud platforms (AWS / Azure / GCP)
VPC, Subnets, NSG rules all use CIDR
Example:
Azure VNet: 10.0.0.0/16
Subnet: 10.0.1.0/24
- Firewalls & Security rules
Allow or deny traffic by CIDR range
Example:
Allow 203.0.113.0/24

CIDR	Subnet Mask	Total IPs	Usable IPs
/32	255.255.255.255	1	1
/31	255.255.255.254	2	2 (point-to-point)
/30	255.255.255.252	4	2
/29	255.255.255.248	8	6

/28	255.255.255.240	16	14
/27	255.255.255.224	32	30
/26	255.255.255.192	64	62
/25	255.255.255.128	128	126
/24	255.255.255.0	256	254
/23	255.255.254.0	512	510
/22	255.255.252.0	1,024	1,022
/21	255.255.248.0	2,048	2,046
/20	255.255.240.0	4,096	4,094
/19	255.255.224.0	8,192	8,190
/18	255.255.192.0	16,384	16,382
/17	255.255.128.0	32,768	32,766
/16	255.255.0.0	65,536	65,534
/15	255.254.0.0	131,072	131,070
/14	255.252.0.0	262,144	262,142
/13	255.248.0.0	524,288	524,286
/12	255.240.0.0	1,048,576	1,048,574
/11	255.224.0.0	2,097,152	2,097,150
/10	255.192.0.0	4,194,304	4,194,302
/9	255.128.0.0	8,388,608	8,388,606
/8	255.0.0.0	16,777,216	16,777,214
/7	254.0.0.0	33,554,432	33,554,430
/6	252.0.0.0	67,108,864	67,108,862
/5	248.0.0.0	134,217,728	134,217,726
/4	240.0.0.0	268,435,456	268,435,454
/3	224.0.0.0	536,870,912	536,870,910
/2	192.0.0.0	1,073,741,824	1,073,741,822
/1	128.0.0.0	2,147,483,648	2,147,483,646

- **/1** → Half of the entire IPv4 internet
- **/2** → One-quarter of all IPv4 addresses
- **/3 to /5** → Used by **regional internet registries**
- **/6 to /7** → Large ISPs / telecom providers

Cloud providers usually limit:

- **VNet/VPC max = /16**
- **Subnet max = /24 or /23** (provider-dependent)

➔ How do IP address are divided ?

[[[Ex: Sir told : IP address into 256 Subnets

But usable are only 254

1st for network ID, last for broadcast ID

But for cloud $254 - 3 = 251$ are usable

1st for network ID, last for broadcast ID

2nd is for gateway, 3rd and 4th for dns

- In cloud In one subnet 251 machines can be installed
- What is Infrastructure size for changing CIDR size ?

Subnet total size 256

Ex : Web server 50

App sever 25

Db server : 10

Ip address : 10.1.0.0/16

WebSubnet : 10.1.1.0/24 => 256

10.1.1.0 /26 => 62

10.1.1.1 to 10.1.1.62

Webserver : 50

Appsubnet : 10.1.2.0/24 => 256

10.1.1.65 /27 =>30

10.1.1.65 to 10.1.1.94

App server : 25

DBsubnet : 10.1.3.0/24 => 256

10.1.1.96 /28 => 14

10.1.1.97 to 10.1.1.110

DBserver : 10

Total of 85 subnets used out of 768

Almost 700 ip address are wasted

Using this <https://www.calculator.net/ip-subnet-calculator.html>

Only usable are divided and rest are left

1.3.3. DNS – DOMAIN NAME SYSTEM

➔ DNS is a global system that translates human-friendly domain names into machine-friendly IP addresses.

Example: When you type www.google.com, DNS converts it into an IP address as 142.250.190.78

➔ Computers communicate using **IP addresses** (numbers),

- but humans prefer **names** (google.com).
- DNS is like the internet's: **Phonebook, Translator, Address directory**
It maps **names** ➔ **numbers** automatically.

➔ How DNS Works (In Detail, Step-by-Step)

- **You type a domain name** (example: facebook.com).
- **Browser checks its cache** for a saved IP.
- **Computer checks local DNS cache.**
- **DNS resolver** (ISP or Google DNS) is asked.
- Resolver checks the **Root DNS servers**.
- Root points to the correct **TLD server** (.com, .org, etc.).
- Resolver asks the **Authoritative DNS server** for the real record.
- **IP address is returned** (facebook.com → 31.13.71.36).
- **Browser connects to that IP**, and the website loads.

➔ DNS Records (Very Important)

DNS stores information in **records**, each serving a purpose:

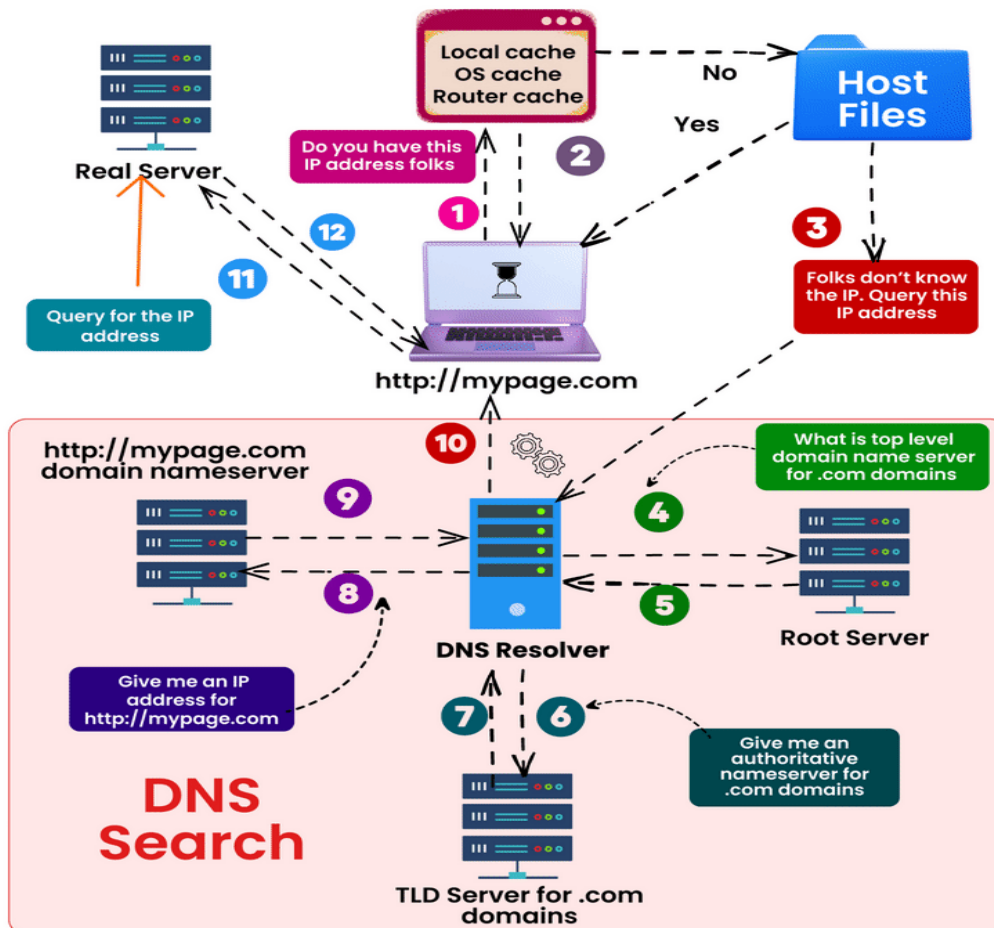
Record	Purpose
A record	Domain → IPv4 address
AAAA record	Domain → IPv6 address
CNAME	Alias (e.g., www → main domain)
MX	Mail servers for emails
NS	Nameservers for the domain
TXT	Text data (security, verification)
SRV	Services like VoIP
PTR	Reverse lookup (IP → domain)

➔ Components of DNS

- I. **DNS Resolver** : First server your computer asks (usually ISP).
- II. **Root Servers** : Top-level servers (only 13 logical root server groups worldwide).
- III. **TLD Servers** : Manage extensions like: .com, .net, .in, .gov, .edu
- IV. **Authoritative DNS Servers** : Hold the actual DNS records created by the domain owner.



HOW DNS WORKS



➔ 🔒 DNS Security Concepts

- DNS has weaknesses and improvements:
 - **DNS Cache** : Stored results to speed up browsing.
 - **DNS Spoofing / Poisoning** : Hacker makes DNS return the wrong IP → dangerous.
 - **DNSSEC** : Security extensions that verify DNS responses.
- If you want to purchase Domain names you can purchase from Godaddy website.
- ❖ DNS is the system that turns website names into IP addresses so computers can find each other.

DNS (Domain Name System) is the system that translates human-readable domain names (like `www.google.com`) into IP addresses (like `142.250.190.78`)

- That computers use to communicate on the internet.

- It is essentially the internet's phonebook.

☑ COMPLETE DNS RECORD TYPES

🌐 ADDRESS & HOST RECORDS

- I. **A** : Hostname → IPv4 address
Example : example.com. A 93.184.216.34
- II. **AAAA** : Hostname → IPv6 address
 Ex. example.com. AAAA 2606:2800:220:1:248:1893:25c8:1946
- III. **A6 (Obsolete)** : IPv6 record with delegation
 Ex. example.com. A6 0 2001:db8::1

🌐 NAME & ZONE CONTROL

- IV. **NS – Name Server** : Authoritative name server
 Ex. example.com. NS ns1.example.net.
- V. **SOA – Start of Authority** : Zone authority & admin metadata
 example.com. SOA ns1.example.net. admin.example.com. 2026010101

🔄 ALIASING & REDIRECTION

- VI. **CNAME – Canonical Host Name** : It is used to make **one domain name an alias of another domain name**.
 Ex : www.example.com. CNAME example.com.
- VII. **DNAME - Delegation Name Record** : It is used to create an alias for an entire DNS subtree.
 Ex : example.com. DNAME example.net.

✉ EMAIL ROUTING & AUTHENTICATION

- VIII. **MX – Mail Server for domain** : It tells the internet **which mail server(s) are responsible for receiving email** for a domain.
 Ex : example.com. MX 10 mail.example.com.
- IX. **TXT – Text Record or Arbitrary text (verification, policies)**
 It stores arbitrary text data and is most commonly used for email security, domain verification, and policy information.
 Ex : example.com. TXT "v=spf1 include:_spf.google.com ~all"
- X. **SPF – Sender Policy Framework** : It is an email authentication method that helps prevent email spoofing by specifying which mail servers are allowed to send email on behalf of a domain.
 Ex : example.com. SPF "v=spf1 -all"
- XI. **DKIM – Domain Keys Identified Mail (via TXT)** : Email signature public key
 It is an **email authentication mechanism** that uses **cryptographic signatures** to verify that an email:
 - Was **authorized by the domain**, and
 - Was **not altered in transit**

Ex : selector1._domainkey.example.com. TXT "v=DKIM1; p=MIIBljANBgk..."

- XII. **DMARC** – Domain-based Message Authentication, Reporting & Conformance (*via TXT*)
DMARC is a DNS record that tells receiving mail servers how to handle emails that fail SPF and/or DKIM checks, and where to send reports.

Simple : Email authentication policy

Ex : _dmarc.example.com. TXT "v=DMARC1; p=reject"

SERVICE DISCOVERY

- XIII. **SRV** – Service Record : Service Location (host + port)
It is used to **define the location of services** (host + port) for a domain.
It's commonly used by **VoIP, SIP, LDAP, Active Directory, XMPP, Minecraft, Kubernetes**, and more.
Ex : _sip._tcp.example.com. SRV 10 60 5060 sip.example.com.
- XIV. **NAPTR** – **Naming Authority Pointer** : Rewrite rules for services
A **NAPTR record** in DNS is used to **rewrite or translate one name into another**, often as part of **service discovery**.
example.com. NAPTR 100 10 "U" "E2U+sip" "!^.*\$!sip:info@example.com!"
- XV. **SVCB** – **Service Binding** : (modern HTTPS replacement)
A **SVCB record** in DNS tells a client **how to connect to a service** for a domain — including **which protocol, port, hostname, and connection parameters to use**.
Why SVCB is used : Traditional DNS records (A, AAAA, SRV) only give **location**.
SVCB gives **connection instructions**.
Ex : example.com. SVCB 1 svc.example.net. alpn=h2
- XVI. **HTTPS** – Hyper Text Transfer Protocol Secure service parameters
In DNS context, HTTPS is a DNS record type, not the protocol itself.
An HTTPS DNS record tells a browser how to connect to a website securely and efficiently.
It is actually a specialized form of SVCB record designed only for HTTPS websites.
HTTPS $\subset$ SVCB
Why HTTPS DNS record is used
Before HTTPS records : Browser queried A / AAAA
Then negotiated protocol later (TLS, HTTP/2, HTTP/3)
With HTTPS records : Browser learns protocols, ports, and parameters in advance
Ex : example.com. HTTPS 1 . alpn=h3

REVERSE DNS

- XVII. **PTR** – Pointer Record : IP $\rightarrow$ hostname
A PTR record in DNS is used for reverse DNS lookup — it maps an IP address back to a domain name.
Normal DNS : Domain name $\rightarrow$ IP address
PTR (reverse DNS) : IP address $\rightarrow$ Domain name
Why PTR is used : Identify the hostname of an IP address, Email servers use it to prevent spam
Used in network troubleshooting & logging
Example : IP address : 8.8.8.8, PTR record : 8.8.8.8.in-addr.arpa. IN PTR dns.google.

34.216.184.93.in-addr.arpa. PTR example.com.

DNSSEC (SECURITY)

XVIII. **DNSKEY – DNS Public Key Record**

A DNSKEY record stores a public cryptographic key used in DNSSEC (Domain Name System Security Extensions).

It helps verify that DNS responses are authentic and not modified.

Why DNSKEY is used : DNS normally has no security against spoofing.

DNSKEY enables DNSSEC, which protects against : DNS spoofing, Cache poisoning, Man-in-the-middle attacks

Types of DNSKEY

Type	Purpose
KSK (Key Signing Key)	Signs the DNSKEY set
ZSK (Zone Signing Key)	Signs other DNS records

Both are stored as DNSKEY records.

Ex : example.com. DNSKEY 257 3 13 AwEAAZ...

XIX. **DS – Delegation Signer**

Meaning: Parent → child DNSSEC link

A DS record is used in DNSSEC to link a parent DNS zone to a child zone securely.

It proves that the child zone's DNSKEY is trusted by the parent zone.

Why DS is used : DNSSEC works using a chain of trust.

DS records are the bridge between zones.

Without DS : DNSSEC chain breaks, DNS validation fails

example.com. DS 2371 13 2 8849AABF...

XX. **RRSIG – Resource Record Signature** : DNSSEC record signature

An RRSIG record contains a digital signature for DNS records in a zone, used by DNSSEC to prove that the data is authentic and unchanged.

“Has this DNS data been tampered with?”

Why RRSIG is used : RRSIG is a core part of DNSSEC and protects against : DNS spoofing, Cache poisoning, Fake DNS responses

Ex : example.com. RRSIG A 13 2 3600 20260201...

XXI. **NSEC – Next Secure Record** : Proves non-existence (DNSSEC)

An NSEC record is used in DNSSEC to prove that a DNS name or record does NOT exist — securely.

“This domain/record truly isn't there, and nobody is lying about it.”

Why NSEC is used : Without DNSSEC, attackers could : Fake “no such domain” responses
NSEC provides authenticated denial of existence, preventing spoofed negative answers.

Ex : example.com. NSEC www.example.com. A MX RRSIG

XXII. **NSEC3 – Next Secure Record version 3** : Hashed NSEC

NSEC3 is a DNSSEC record used to prove that a domain name or record does NOT exist, without exposing actual domain names.
It does the same job as NSEC, but more securely.

Why NSEC3 is needed

NSEC problem: : Allows zone walking (anyone can list all domain names)

NSEC3 solution : Uses hashed domain names, Makes zone enumeration much harder

Ex : example.com. NSEC3 1 0 10 ABCD1234...

XXIII. **NSEC3PARAM** – Next Secure Record version 3 Parameters

An NSEC3PARAM record tells how NSEC3 should be used in a DNS zone.

It defines the hashing settings that resolvers need to understand NSEC3 proofs.

Why NSEC3PARAM is used

NSEC3 uses hashing, and those hashes depend on parameters like : Which hash algorithm, How many iterations, What salt value

NSEC3PARAM publishes these parameters so validators know how to verify non-existence.

Ex : example.com. NSEC3PARAM 1 0 10 ABCD1234

CERTIFICATES & TRUST

XXIV. **CAA** – Certification Authority Authorization : Allowed SSL certificate issuers

A CAA record in DNS lets a domain owner control which Certificate Authorities (CAs) are allowed to issue SSL/TLS certificates for their domain.

“Who is allowed to give HTTPS certificates for this domain?”

Why CAA is important

Without CAA : Any trusted CA could issue a certificate for your domain

With CAA : Only approved CAs can issue certificates, Helps prevent mis-issued or fraudulent certificates

Example CAA record

example.com. IN CAA 0 issue "letsencrypt.org"

example.com. CAA 0 issue "letsencrypt.org"

XXV. **TLSA** – Transport Layer Security Authentication : DANE TLS certificate binding

A TLSA record in DNS is used by DANE (DNS-based Authentication of Named Entities) to verify TLS certificates using DNSSEC instead of (or in addition to) traditional Certificate Authorities.

“Which TLS certificate or key should this service use?”

Why TLSA is used

Traditional TLS relies on public CAs.

TLSA + DNSSEC allows : Stronger security, Protection against rogue CAs, Certificate pinning via DNS

Ex : _443._tcp.example.com. TLSA 3 1 1 abcdef...



NETWORK & SYSTEM INFO

XXVI. **HINFO** – Host Information : Host hardware & OS

An HINFO record in DNS stores information about a host's hardware (CPU) and operating system.

“What kind of machine is this host running on?”

HINFO has two text fields : CPU type, Operating System.

Why HINFO was used : Network diagnostics, Debugging and system identification

Early internet documentation

Why HINFO is rarely used today : Security risk (reveals system details), Not needed for modern networking, Most DNS servers block or avoid it

Example : example.com. IN HINFO "INTEL-64" "LINUX"

example.com. HINFO "Intel" "Linux"

XXVII. **LOC** : Location : Geographic location

A LOC record in DNS is used to specify the geographical location of a domain, server, or host. It provides latitude, longitude, altitude, and other location details.

“Where in the world is this server?”

Why LOC is used : Mapping servers geographically, Network optimization (routing, latency-based decisions), Informational purposes

LOC record format : example.com. IN LOC 37 47 0.000 N 122 25 0.000 W 10.00m 20.00m 100.00m 10.00m

example.com. LOC 37 48 46 N 122 24 9 W 10m

Fields explained

Field	Meaning
Latitude	Degrees, minutes, seconds, N/S
Longitude	Degrees, minutes, seconds, E/W
Altitude	Height above sea level in meters
Size	Diameter of host or area (optional)
Horizontal Precision	Precision of location
Vertical Precision	Vertical accuracy

XXVIII. **RP** - Responsible Person

An RP record in DNS specifies the email address of the person responsible for a domain or host.

“Who should be contacted about this domain?”

Why RP is used : Administrative contact for domain, Mainly informational, Rarely used today because WHOIS provides this info

How RP works : The RP record points to another domain that contains a TXT record with the email.

The email address uses . instead of @

Example: admin.example.com. → corresponds to admin@example.com

First field → mailbox domain

Second field → text domain containing additional info

```
example.com. RP admin.example.com. hostmaster.example.com.
```

XXIX. **AFSDB** – Andrew File System Database

An AFSDB record in DNS is used to specify a server that provides services for the Andrew File System (AFS) or related services.

“Which server handles AFS or DCE/DFS services for this domain?”

Why AFSDB is used

Originally for Andrew File System (AFS), a distributed file system

Can also be used to locate DCE (Distributed Computing Environment) / DFS services

Helps clients find the right server for certain network services

Meaning: AFS database servers

```
example.com. AFSDB 1 afsdb.example.com.
```

 HISTORICAL / RARE / OBSOLETE

XXX. **MB** - Mailbox Domain Name

An MB record in DNS was used to identify a host that has a specific mailbox.

“On which host does this user’s mailbox live?”

Why MB existed : Early email systems needed a way to locate user mailboxes

Part of the original DNS mail-related records

Example : user.example.com. IN MB mailhost.example.com.

Meaning : The mailbox for user@example.com is located on mailhost.example.com.

```
example.com. MB mailbox.example.com.
```

XXXI. **MG** – Mail Group

An MG record in DNS was used to define a group of mailboxes — similar to a mailing list.

“Which mailboxes belong to this mail group?”

Why MG existed : Early DNS mail routing, Simple mailing list functionality

Example : group.example.com. IN MG user1.example.com.

group.example.com. IN MG user2.example.com.

Meaning : Mail sent to group@example.com would go to user1 and user2.

```
example.com. MG group.example.com.
```

XXXII. **MR** – Mail Rename

An MR record in DNS was used to rename or redirect one mailbox name to another mailbox.

“If mail is sent to this mailbox, which mailbox should actually receive it?”

Why MR existed : To handle mailbox renaming, To support user name changes

Example : olduser.example.com. IN MR newuser.example.com.
Meaning : Mail sent to olduser@example.com would be delivered to newuser@example.com.
example.com. MR newmail.example.com.

DNS Record	Full Form	Purpose	Current Status
MB	Mailbox Domain Name	Specifies the host where a user's mailbox is located	✗ Obsolete
MX	Mail Exchange	Specifies mail servers responsible for receiving email for a domain	☑ Actively used
MG	Mail Group	Defines a group of mailboxes (mailing list-style)	✗ Obsolete
MR	Mail Rename	Redirects mail from one mailbox name to another	✗ Obsolete
MF	Mail Forwarder	Specifies a mail-forwarding host	✗ Obsolete

XXXIII. **NULL** – Null Resource Record (*testing*)

A NULL record in DNS is an experimental record type used to store arbitrary binary data.
“Store some raw data here — DNS doesn't interpret it.”

Key characteristics : Holds opaque (raw) data, DNS does not process or understand the content, Meant for experimentation and testing

Why NULL existed : To experiment with new DNS uses, To test how DNS handles unknown data, Placeholder for future ideas

Example : example.com. IN NULL 0x0123456789ABCDEF
(The data is just bytes; its meaning depends on the application.)
example.com. NULL \# 0

XXXIV. **WKS** – Well Known Services

A WKS record in DNS was used to list which network services (ports & protocols) are available on a host.
“Which services are running on this server?”

A WKS record includes : IP address, Protocol (TCP or UDP), Bitmap of well-known ports (0–1023)

Why WKS existed : Early service discovery, Network diagnostics, Documenting available services

Example : example.com. IN WKS 192.0.2.10 TCP (HTTP FTP SMTP)
Meaning : Server supports HTTP (80), FTP (21), SMTP (25)
example.com. WKS 192.0.2.1 TCP FTP HTTP

XXXV. **X25** – X.25 Network Address Mapping

An X25 record in DNS was used to map a domain name to an X.25 network address.

What is X.25 : An old packet-switched networking protocol, Widely used before the modern Internet, Used in banking, telecom, and government networks

Why X25 record existed : To support X.25 networks alongside IP, Name-to-address resolution for legacy systems

Example : `example.com. IN X25 "311061700956"`

Meaning : The X.25 address for `example.com` is 311061700956

`example.com. X25 311061700956`

XXXVI. **ISDN** – Integrated Services Digital Network

An ISDN record in DNS was used to map a domain name to an ISDN telephone number.

“What ISDN number is associated with this host?”

An ISDN record may include : ISDN number (E.164 format), Optional subaddress

Why ISDN record existed : To support ISDN-based communication, Early integration of telephony and DNS

Example : `example.com. IN ISDN "123456789" "004"`

"123456789" → ISDN phone number

"004" → Subaddress (optional)

`example.com. ISDN 150862028003217`

PROVIDER-SPECIFIC (NON-RFC)

XXXVII. **ALIAS** – Apex CNAME alternative (**CNAME** = Canonical Name)

In DNS, an **Alias** usually refers to a **CNAME** record.

An **alias** in DNS means **one domain name points to another domain name** instead of directly to an IP address.

“**This name is just another name for that domain.**”

Example : `www.example.com. IN CNAME example.com.`

Meaning : `www.example.com` is an **alias**

`example.com` is the **real (canonical) name**

DNS then resolves `example.com` to its IP using **A / AAAA** records.

How alias (CNAME) works : Alias name → CNAME → Real domain → A / AAAA → IP

Alias is used : Point multiple names to the **same service**, Easy **management** (change IP in one place), Common with **CDNs and cloud services**

Important rules

A **CNAME** **cannot coexist** with other records (A, MX, etc.) for the same name

Cannot be used at the **zone apex** (root domain like `example.com`)

👉 (Except via special records like **ALIAS / ANAME** at DNS provider level)

Alias vs A record

Record	Points to	Use case
--------	-----------	----------

A	IP address	Direct mapping
---	------------	----------------

CNAME (Alias)	Another domain	Name-to-name mapping
---------------	----------------	----------------------

Meaning : `example.com. ALIAS app.hosting.com.`

XXXVIII. **ANAME – Address Name** : Auto-resolved CNAME

An **ANAME record** is a **non-standard (DNS provider-specific)** record that lets you point a **domain name to another domain name**, but **returns IP addresses like an A/AAAA record. "CNAME behavior, but allowed at the root domain."**

Why ANAME exists

Standard **CNAME** records : Cannot be used at the **zone apex** (`example.com`)

ANAME solves this by : Resolving the target domain internally, Publishing **A / AAAA records automatically**

Ex : `example.com. ANAME lb.cloudprovider.net.`

✚ META / INFRASTRUCTURE

XXXIX. **OPT – Option**

An **OPT record** is a **pseudo DNS record** used by **EDNS (Extension Mechanisms for DNS)** to **extend DNS capabilities**.

EDNS0 options (invisible record)

pseudo-record used internally

It just carries **extra options and metadata** between DNS clients and servers.

Why OPT is used

Classic DNS had limits : UDP size limited to **512 bytes**, No place for extra flags or options

OPT + EDNS solves this by allowing : Larger UDP packet sizes, DNSSEC support, Extended flags and features

Important facts : **Not stored in zone files**, Appears only in **DNS queries/responses**, Record type number: **41**, Always at the **additional section**

What OPT carries

Field	Purpose
UDP payload size	Allows >512 bytes
EDNS version	Protocol version
Flags	DNSSEC OK (DO) bit
Options	Extra features (cookies, padding, etc.)

☒ FINAL COUNT

Total covered: Standard RFC records

- DNSSEC records
- Email authentication
- Modern HTTPS/SVCB
- Obsolete & experimental
- Provider-specific records

2. AZURE

- 2.1 Resource Group
- 2.2 Basic Networking
 - 2.2.1 IP Address
 - 2.2.2 CIDR – Count Less Domain Routing
 - 2.2.3 DNS – Domain Name System
 - 2.2.4 VNET – Virtual Network
- 2.3 How to Create Azure Account (Refer to Azure account create and practice Word or PDF)
- 2.4 VNet Peering – Virtual Network Peering
- 2.5 NSG – Network Security Group
 - 2.5.1 NSG in NIC Level
 - 2.5.2 NSG in Subnet Level
- 2.6 ASG – Application Security Group
- 2.7 Azure Load Balancer
- 2.8 Azure Application Gateway
- 2.9 Traffic Manager
- 2.10 App or Application Service
- 2.11 AFD – Azure Front Door
- 2.12 Storage Account
- 2.13 Identity management (Storage Account Access)
- 2.14 Image Creation
- 2.15 VMSS – Virtual Machine Scale Set
- 2.16 Azure Bastion
- 2.17 RSV – Recovery Service Vault

- 2.18 RSVR – Recovery Service Vault Recovery
- 2.19 ASQL – Azure Structure Query Language
- 2.20 Hub and Spoke
- 2.21 AVPN – Azure Virtual Private Network
- 2.22 AAD – Azure Active Directory
- 2.23 AM – Azure Monitor
- 2.24 ALA – Azure Logic Apps

3. Terra Form

3.1 Intro